SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

[e.g. This section "YFood" is introducing the strategic solution of the identified problem statement highlighted in Product Review Documentation.]

1.1. Purpose:

[e.g. The system analysis document highlights the technical scope and design-related decision behind the YFood development.]

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *[e.g. It's a cloud native platform. The services are separated into Microservices. Deployed on AWS and these microservices work independently.]*
2. ***Data Flows:***
    a. *[e.g. How the data of Order of Food flows from Customer's mobile/devices, moves through backend API, gets stored in PostgreSQL in persistent manner and finally returns a confirmation message for the user.]*
3. ***Model Process:***
    a. *[e.g. The model shows the end-to-end workflow of the core features: placing an order, assigning a rider for that specific order and confirming the delivery.]*
4. *Role of Reference:*
    a. *[e.g. This document is a reference for all the developers and testers to enable the overall high-level architecture of this project.]*

*1.2. **Background**:*

[e.g. Company Y had a wide network of drivers with one of their subsidiaries. They have found a market where the order and delivery of the food is quite manual.]

1. *Previous Version:*
    a. *[e.g. The existing subsidiaries are operated into Mobile Platform (Native) in a one single application. It's convenient for users to get the benefits of each service from a single platform. One is "E-hailing" service, and another one is "Payment Network".]*
2. *Changes in Major Architectural Components*
3. *New capabilities that are only introduced in this*

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Customer	*Browse the food menus of different vendors, places order and tracks the order using the mobile platform*	*Simple Food order experience, payment by YFin (Subsidiary – Payment Network).*
Rider	*Receiving the delivery tasks and fulfil the delivery of correct order*	*Clarified details of the order, easier navigation system and notification service for newer tasks*
Vendors	*The vendors will be receiving food orders as per their menu, prepares and handover to delivery rider*	*Immediate notification of the food ordered correct order information.*
Development Team	*Build the platform and maintain it*	*Clear API usage contracts, codebase is modular and well-documented architecture*
QA Team	*Validating the system behavior for all the user roles*	*Defined scope, the request and response formats, and coverage of edge cases*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*[e.g. Mobile App (iOS & Android)]*
Architecture	*[e.g. Cloud-Server with Microservices]*

[e.g. YFood is primarily structured as a client-server-based system in which there are 3 types of clients. 1. Customer Mobile Application, Rider Mobile, AND Vendors Web-based Dashboard. All these clients communicate by Backend API layer. So, three independent microservices: Order Service, Rider Assignment Service and Notification Service. All services are deployed on Amazon Web Services, and all share a PostgreSQL database that is hosted also in AWS RDS.]

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic "AI Module". The following Dependency diagram includes:

2.1.3 Dependency Diagram

a. *How the prompts are being constructed and sent to the GLM API*
b. *What goes into the context window?*
c. *How is your system receiving, parsing the responses and passing to the next system component?*
d. *Where the limitation of token are enforced or input are chunked before it actually reaches to GLm*
e. *Also, do outline all the major API calls between the system components [e.g. Customer Mobile App calls "Amazon API Gateway" for Order Service]*
f. *Do show how frontend, backend, services, database and external API interacts with each other [e.g. Order service communicates with Amazon RDS to ensure persistence of order data]*
g. *Must highlights the distributed components if the system is planning to be deployed across multiple services*

2.2.2. Sequence Diagram

a. *Here, do walk through one user interaction flow or customer journey with the stakeholders [e.g. Place Order Flow]*

© UMHackathon 2026

b. *Show the interactions step by step.*
c. *Do demonstration of each key system features does operate from start to finish*

2.2. Technological Stack

2.2.1. Frontend [e.g. React Native – Cross platform needed for the user base of iOS & Mobile]
2.2.2. Backend [e.g. FastAPI – RESTful API for the microservices]
2.2.3. Database [e.g. PostgreSQL – Relational Database for orders, users, vendors, and riders]
2.2.4. Cloud/Deployment [e.g. AWS EC2 (Server), RDS (PostgreSQL), S3 (Static Assets)

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

a. *Illustrate the data movement across the entire system that includes external entities (users), processes, external service and databases*
b. *Show how proper laws are met (e.g. Black Hole)*

2.3.2. Normalized Database Schema

a. Show the logical data model (ERD) with Database designed to 3NF form to ensure data integrity and relationships. [e.g. order items is a junction table that is resolving the M: N relationship between orders and menu items.

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. *Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.* ***[e.g. in Table]***

#	*Features*	*Description*
1	*Customer Order Placing*	*Customers browse the menu of different vendors, choose the items as per preferred quantity, add to the card and pay accordingly.*
2	*Rider Assignment*	*After confirmation of payments, riders are being located based on distance and location.*

© UMHackathon 2026

3	*Vendors Notification*	*Vendors receive digital based notification with order details displaying the order-related info.*

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Scalability	*[e.g YFood system must handle the orders especially during lunch hours and peak hours.]*	*[e.g. Use AWS ASG for scaling Servers and ELB for distribution of traffic.]*
Reliability	*[e.g. Placement of orders should have 99% uptime, and failed payment will not create any orphaned/completed orders]*	*[e.g., payment confirmation triggers database INSERT SQL query for orders.]*
Maintainability	*[e.g. Each service like (Order, Rider, Vendor) is separated and deployed individually.]*	*[e.g. Independent Pipeline for each service during CI/CD stage with its own configurations.]*
Token Latency	*[e.g. The API calls to GLM must return a response within 3s under normal load but at p95]*	*[e.g. Timeout of 5s for Async API calls. Displaying user-facing loading state while awaiting for response]*
Cost Efficiency	*[e.g. Average costs of token per user session must not exceed a defined budget threshold.]*	*[e.g. The length of the prompts should be fixed length. The inputs from the user are truncated or chunked before injecting and the repeating queries are cached for 24 hours.]*

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026. [e.g. below]

a. *During delivery, the In-App chat features between the rider and customer*
b. *Promotional offers, discount code, vouchers*
c. *Scheduled Delivery (Order now, and wants delivery to be delivered on a specific time)*

© UMHackathon 2026

4. *Monitor, Evaluation, Assumptions & Dependencies*

4.1. *Technical Evaluation:*

4.1.1. ***Grayscale Rollout & A/B Testing****: Do highlight team's strategy of deployment [e.g. new feature of user-preferred food menu are released to 5% of real users initially, monitoring and then gradually increased until 100% if no issues are detected. After it passes, A/B testing is conducted to determine which variants of two strategies perform better.]*

4.1.2. ***Emergency Rollback (ER) & Golden Release****: Do mentioned the strategies for Emergencies. [e.g. In terms of recovery & traffic management for YFood system, if there are anomalies, ER is triggered. The automated pipeline does deploy the Golden Version of the application.]*

4.1.3. *Priority Matrix:*

4.1.3.1. *Clearly define the Priority Matrix and what are the trigger conditions and what actions will be taken for your project. [e.g. P1 Critical: If payment failure error is more than 1%, emergency rollback to Golden release.]*

4.2. ***Monitoring****:*

4.2.1. *Show the planning for your monitoring plan of the system and how it will trigger if any damage is detected of the health of the system.*

4.3. ***Assumptions:*** *State key operational & environmental conditions that you assumed to be valid while development of system and deployment. [e.g.*

- ***a.*** *Users have stable internet connections on their devices during the placement of orders*
- ***b.*** *Restaurants have their own devices to access the web-dashboard for monitoring of orders]*

4.4. ***External Dependencies:*** *Do list all the external tools that are applied to your system to function. [e.g. below]*

Tools	*Purpose*	*Risks*

© UMHackathon 2026

Z Ai GLM API	*It should be the core LLM reasoning engine - that will handle the prompt inference, multi-step reasoning and generating structured output*	*High- in case of GLM API rate limitation - core product will fail. So, implement a mitigation strategy [e.g. graceful degradation state from users]*
Google Maps API	*Tracking rider location and optimize the route*	*Medium level- if real time tracking is being disabled.*

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. ***Project Timeline:*** *Show the timeline of development and justify accordingly [e.g. Day 1-2, Project Planning etc.]*

5.2. ***Team Members Role:*** *Highlight the role of team members [e.g. Person A – Role Backend Lead – Fast API Microservices & Schema Design]*

5.3. ***Recommendations:*** *If any recommendation is applicable in terms of scalability, performance and reliability [e.g. using Redis Cache for Menu data]*

© UMHackathon 2026